

Lab 5: Voltmeter with ADC, DAC, and PDB implementation

Introduction

This lab is intended to apply our knowledge of interrupts and build on lab four's circuit to create a voltmeter. To understand DAC and ADC within the board two projects will be analyzed to understand simple implementation of these concepts and how to apply them to the created circuit.

The first implemented requirement is to input a voltage and display it on the 7 segment display. The 7 segment display must also be programmed with interrupts as a constraint meaning lab four's code will have to be modified. The code implementation must use an ADC and DAC in order to communicate data from voltage, into the 7 segment display.

The second development will be using the function generator to plot inputs and outputs. These graphs will be analyzed to determine conversion time, as well as other components such as maximum input frequency with allowable error. Another area of interest will be the average output from the circuit and plotting these values.

Background

An oscilloscope is a tool used to analyze electrical signals. Its useful for an indepth look at electrical outputs and inputs from circuits and comparing amplitudes, periods and all characteristics of the graph. These graphs are displayed and can be moved, zooming in and out, and other functions to allow for close up look at these functions. Many oscilloscopes also have math functions allowing to add or subtract waves as well as finding mean graphs and other combinational waves.

A function generator is used to generate a wide variety of signals, with different shapes, and frequencies. This customization allows for a wide variety of different signals to be generated and fed into whatever system or circuit the user desires.

An Analog Digital Converter (ADC) is used to convert signals from analog, to digital or discrete. Similarly a Digital Analog Convert (DAC) converts the other way. These components are commonly used within electronic devices to convert different components allowing them to read each other's data.

A programmable delay block (PDB) allows to precisely set clock intervals on given equipment. Similar to other timers and interrupts used in the course it has precise programmable intervals at the cost of more power than the LPT.

Voltmeters are used to measure voltage across or at areas of circuits. These are widely used in throughout the electronic industry. More commonly volt meters can be found within multimeters which are capable of measuring current, and resistance which are vital for circuit design.

Periodic interval timer (PIT) is a timer used on the K60 board. This timer draws more power than the LPT but has many overlapping applications. The main advantage of this timer is its precise timing and accuracy being much better at achieving a stable clock cycle.

The 7-segment display given must be connected to the breadboard with resistors in order for the k60 to operate on it. The 7-segment display has 7 inputs for the numbers, each input being only a line which makes up numbers when configured properly. There are also 3 inputs to program the 3 sets of 7 segment displays on the part. There is one final input used to control a decimal point.

Interrupt Requests (IRQ) are signals sent to and from the processor. These can be used in many ways, such as delaying or stopping operations on the flag and most commonly exception handling. When an interrupt is called the processor will stop its current task and put priority on the interrupt unless a higher priority interrupt is in progress.

Nested Vector Interrupt Controller(NVIC) has many applications but in this context is used to control interrupts. In assembly, it is accessed by its memory by storing certain values effectively switching designated bits. Although you can access all interrupts this way the lab only requires turning on the LPT and allowing IRQ's

Keil is a software development IDE used to develop embedded applications. Keil was the main software used for the lab alongside PuTTY and the TWR-K60D100M hardware. Keil was important for the lab because it was needed for the development of the timer loop delay program. It also allows for debugging and porting of the program to the specific hardware used.

TWR-K60D100M was the tower-compatible microcontroller module featuring the MK60DN512VMD10 microcontroller based on the ARM Cortex-M4 architecture. The board was used to load and execute the code that was written for the lab. It was to load programs to and be able to execute and display them on the PuTTY terminal.

Debugging in computer programming is a process that is used not only to identify but also to fix issues or errors in code that are appearing in a program. Debugging was relevant for the lab because looking for errors and analyzing memory and registers was needed when fixing the error in the first segment of the lab as well as helping in the development of the timer loop program.

Registers are the building blocks of ARM assembly. Registers have 32 bits of storage holding information to manipulate within the programs. The program build focuses on using registers one to five for calculations as well as loading and storing data.

Understanding memory locations and how to view memory in Keil was an important part of lab one. Memory is where all files, instructions, and variables are stored. Keil was used to locate addresses in the memory so that the addresses of specific variables and programs could be found and referenced. Understanding where each register was in the memory allowed for an understanding of what was happening in the memory at each register.

Methodology

For requirement one, both of the demo projects were analyzed and read over to gain an understanding of the ADC and how the DAC works. This was done by building both programs and downloading them to the board, followed by a deep analysis of the code in each file.

Requirement two involved the development of a voltmeter that could measure DC voltage values between 0-3.3V and using an analog line to input the DC voltages and display them on the 7-segment display developed in the last lab. This was done by utilizing the same circuit setup from lab 4 and modifying it to have a voltage input line connected to the B40 pin for ADC0. Code from lab 4 was also reused and modified by adding the ADC to the existing code. Following the lab 5 materials the DAC and PDB were configured. The SIM clocks were first all initialized for PIT, ADC, DAC, PDB and all the necessary GPIO ports. The PDB was configured by resetting PDB0_MOD to zero and enabling interrupts, PDB ADC pre trigger outputs, and pulse output. Pulse was set to one after x number of pulses, and prescaler set to 0 & multifactor to 1. The ADC was configured for long sample times for both CFG1 and CFG2. The default sample time was set, hardware trigger selected, enabled hardware function and continuous conversion, and selected input for voltage (ADC0). To display the current voltage to the 7-segment display the average voltage (explained in more detail in requirement three and Figure 1) was taken and converted to volts, this was then displayed by utilizing the division by ten algorithm. After completion of code and circuit the correct voltage values were displayed).

Requirement three builds on what was done for requirement two. For this requirement instead of a constant voltage input a function generator to generate a 100 Hz square wave with voltage between 0-3.3V connected to the same B40 pin and the implementation of the DAC was required. The program would display the mean, or average, voltage on the 7-segment display. The circuit was slightly modified to include an output for the DAC connected to the A32 pin (DAC0), the full circuit shown in Figure 2. The DAC was implemented by clearing DAC0_DAT0L, DAC0_DAT0H, SR, C0, and C1. C2 was then loaded and the DAC enabled. The handling/implementation of the ADC, DAC, PDB, and display/calculation of the average voltage is explained in Figure 1 shown below.

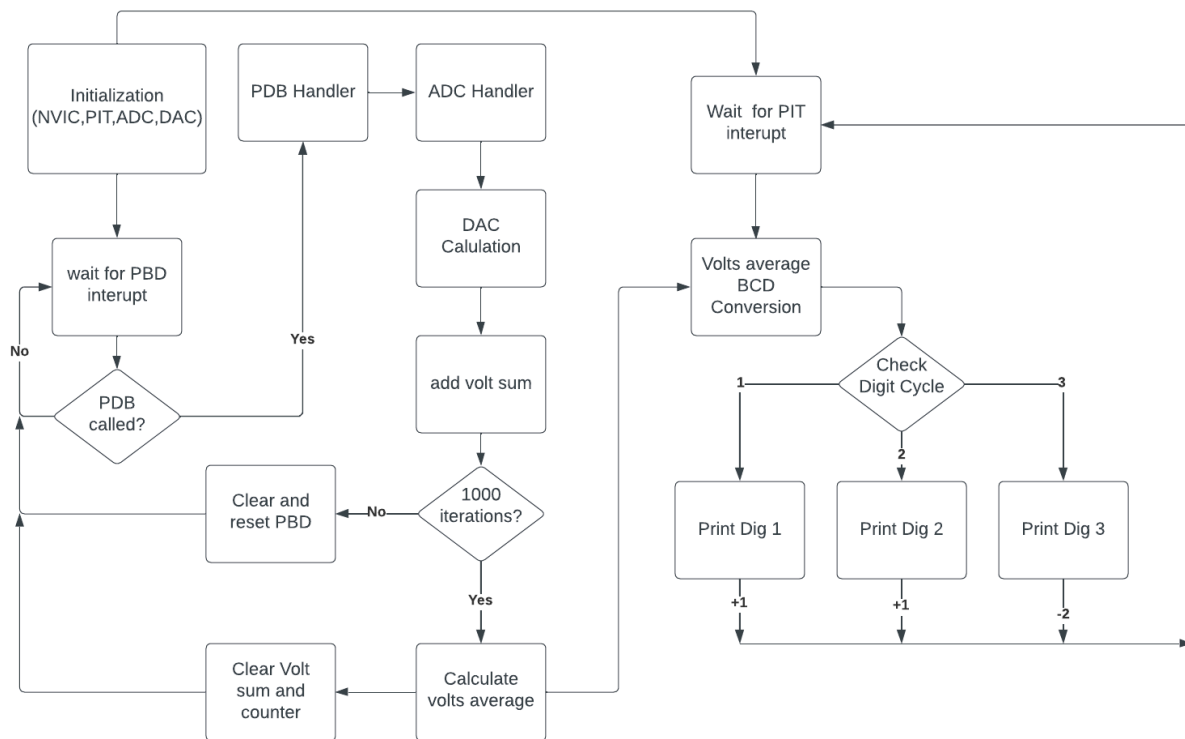


Figure 1: Flow chart of requirement three code

The circuit was then connected to the function generator to give input to ADC0, channel one of the oscilloscope was connected to the DAC output and channel two connected to the signal from the function generator. The function generator was turned on with specified conditions and the DC power supply connected to give the display power. The results of which are shown below in Figure 3. The MATH button on the oscilloscope was used to obtain the difference of the two symbols, this was found to be [65.5mV] and shown in Figure 4. The percentage error of the DAC was found using

$$\text{Percentage error} = \frac{|V_{\text{observed}} - V_{\text{true}}|}{V_{\text{true}}} \times 100\%.$$

With $V_{\text{true}} = 1.56\text{V}$ and $|V_{\text{observed}} - V_{\text{true}}| = 0.0655\text{V}$ from Figure 4. This gave an error of about 4%, which is considerably small. This error is related to the DAC conversion whether it is offset error or a gain error. The error associated with DAC as well as imperfection caused by the general use of the lab equipment and circuit elements will definitely cause a difference in values. It was observed by increasing the frequency that the output signal begins to break down at around 200 Hz and then becomes most noticeable when greatly increased 500+ Hz.

Conclusion

For this lab all requirements were completed with a successful demonstration. Requirement one was accomplished by testing and gaining a solid understanding of the example ADC and DAC project so that they could be utilized in requirements two and three.

Requirement two was accomplished by utilizing the ADC, PDB and DAC alongside code from the previous lab to display a voltage value on the 7-segment display. This allowed for a deeper understanding of how the ADC system works and how it is used with the PDB. This section shows how the device can be interfaced and used in a real life system, in this case taking a voltage and displaying it.

The third requirement was to implement a DAC into the code and have this displayed on the oscilloscope alongside the input function generator signal. This allowed for further understanding of how a DAC works as it allowed for the DAC and input signal to be displayed and the differences between the two analyzed. This was successfully completed with all expected outputs being shown.

References

- [1] Radu Muresan,(University of Guelph, Canada).ENGG3640: Microcomputer Interfacing Laboratory Manual, F22, (August 2015). Accessed: Sep. 29th 2023. [Online]. Available: <https://courselink.uoguelph.ca/d2l/le/content/833041/viewContent/3542615/View>

Appendix

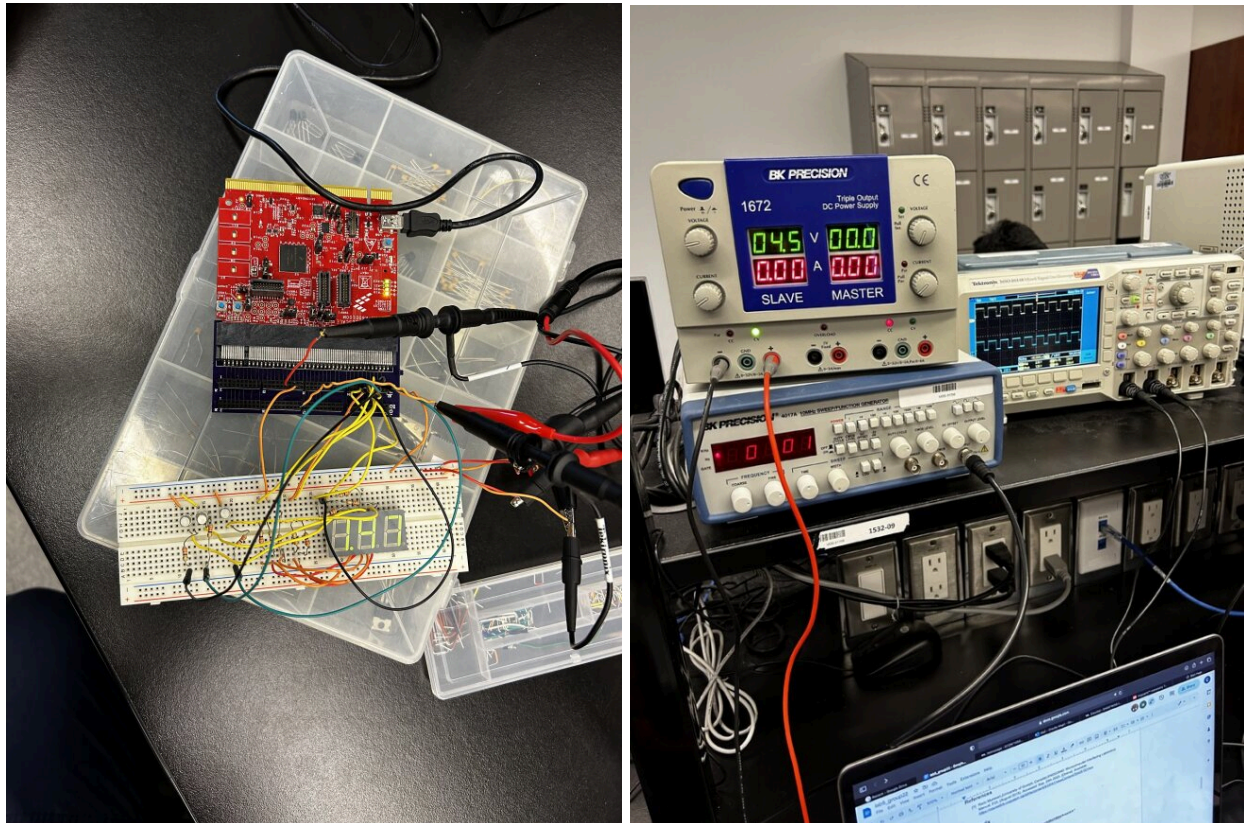


Figure 2: circuit set up for requirement three

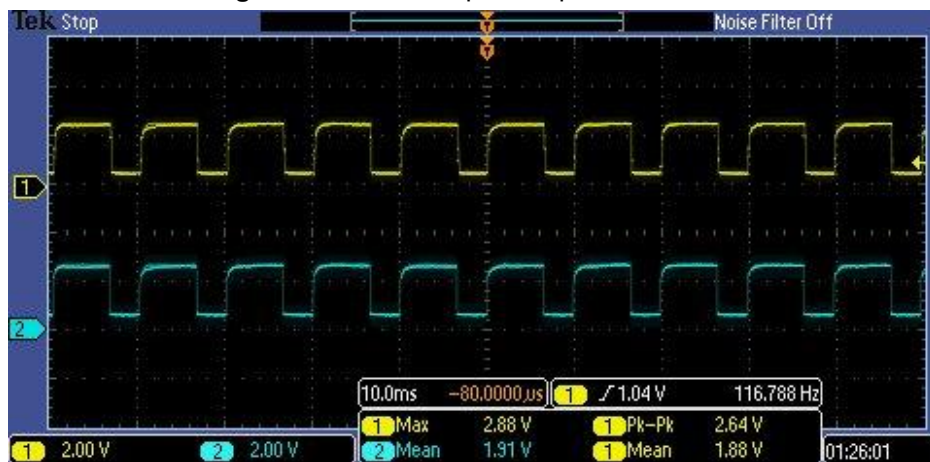


Figure 3: oscilloscope image of working DAC



Figure 4: oscilloscope image of working DAC with MATH visible

Part 3 code

```
#include <stdio.h>
#include "C:/Freescalar/KSDK_1.2.0/examples/twrk60d100m/board.h"
#include "fsl_debug_console.h"
#include "math.h"

// display variable
int ledCycle = 0;
int zero = 192UL;
int one = 249UL;
int two = 164UL;
int three = 176UL;
int four = 153UL;
int five = 146UL;
int six = 130UL;
int seven = 248UL;
int eight = 128UL;
int nine = 152UL;
int a = 136UL;
int b = 131UL;
int c = 198UL;
int d = 161UL;
int e = 134UL;
int f = 142UL;
int decimal = -128UL;
int digit3 = 256UL;
int digit2 = 512UL;
int digit1 = 1024UL;

// ADC handler
int counter = 0;
```



```

int voltsHex = 0;
int voltsHexShifted = 0;
int voltsSum = 0;
int voltsAvg = 0;
int voltsDigits[3] = {0,0,0};

__INLINE static void Intialize(void) {
    SIM->SCGC6 = (1UL << 23 | 1UL << 27 | 1UL << 22);    // PIT, ADC, PDB
clock enabled
    SIM->SCGC5 = (1UL << 11 | 1UL << 9);                // enable clock gate to
port A/C
    SIM->SOPT7 = 0x0;                                    // PDB for ADC0
    SIM->SCGC2 = 1UL << 12;                              // enable clock for DAC0

    for(int i = 0; i < 16; i++) {
        PORTC->PCR[i] = 1UL << 8;    // enable GPIO for PCR pins
    }

    PORTA->PCR[7] = 0x00;    // set pin as analog input
    PTC->PDDR = 2047UL;    // set pins PTC 0-10 as output

    //PIT config
    PIT->MCR = (0UL << 1);
    PIT->CHANNEL[0].LDVAL = 2559;    // create interrupt
    PIT->CHANNEL[0].TCTRL = 3UL;    // start interrupt
    PIT->CHANNEL[0].TFLG = 1UL;    // clear interrupt

    //PDB config
    PDB0->MOD = 0x1000;    // reset to
zero
    PDB0->IDLY = 0x0;    //
enable interrupts
    PDB0->CH[0].C1 = 0x00000101;    // enable PDB ADC pre trigger
outputs
    PDB0->POEN = 0x01;    // enable
the pulse output
    PDB0->PODLY[0] = 0x0101000;    // pulse to one after #
pulses
    PDB0->SC = 0x00000f83;    // prescalar to 0
& multifactor to 1

    // ADC config
    ADC0->CFG1 = 0x7d;    // enable bus clock, 16 bit conversion, sample time,
sets divide ratio, clock rate
    ADC0->CFG2 = 0x00;    // default sample time
    ADC0->SC2 = 0x40;    // select hardware trigger

```



```

ADC0->SC3 = 0x0c;      // enable hardware function and continuous conversion
ADC0->SC1[0] = 0x4a;    // select input (AD10)
PDB0->SC |= 0x00010000; // PDB

// DAC config
DAC0->DAT[0].DATL = 0x0; // clear
DAC0->DAT[0].DATH = 0x0; // clear
DAC0->SR = 0x0;         // clear
DAC0->C0 = 0x0;         // clear
DAC0->C1 = 0x0;         // clear
DAC0->C2 = 0x0f;        // load
DAC0->C0 = 0xf0;        // enable DAC

//NVIC setup
NVIC_EnableIRQ(PIT0_IRQn); // enable PIT handler
NVIC->ISER[1] |= 0x02000000;
NVIC->ISER[2] |= 0x00000100;

}

// PIT timer handler
void PIT0_IRQHandler(void) {
    const int convertToBCD[10] = {zero, one, two, three, four, five, six, seven,
    eight, nine};

    PIT->CHANNEL[0].TFLG = (1UL << 0); // reset flag
    if(ledCycle == 0) { //LED 1
        PTC->PCOR = (4095UL); // clear
        PTC->PDOR = (convertToBCD[voltsDigits[2]] + digit1); // set
        ++ledCycle;
    } else if(ledCycle == 1) { //LED 2
        PTC->PCOR = (4095UL); // clear 1
        PTC->PDOR = (convertToBCD[voltsDigits[1]] + digit2); // set
        ++ledCycle;
    } else { //LED 3
        PTC->PCOR = (4095UL); // clear 1
        PTC->PDOR = (convertToBCD[voltsDigits[0]] + decimal + digit3); // set
        ledCycle = 0;
    }
}

// ADC handler
void ADC0_IRQHandler(void) {
    int voltsValue;
    voltsHex = ADC0->R[0]; // read 16 bit input ADC

```

```

        // DAC calculation
        voltsHexShifted = (voltsHex >> 4);           // convert to 12 bits
        DAC0->DAT[0].DATL = voltsHexShifted & 0xff;   // store 8 to lower region
        voltsHexShifted = voltsHexShifted >> 8;       // convert to 4 bits
        DAC0->DAT[0].DATH = voltsHexShifted & 0xf;    // store 4 to higher region

        voltsSum += voltsHex;
        counter++;

        if(counter == 0x1000) {
            voltsAvg = voltsSum / 0x1000;             // calculate average
voltage
            voltsValue = (voltsAvg * 330) / (pow(2, 16) - 1); // convert 16 bit ADC
value to volts
            voltsDigits[0] = (voltsValue / 100) % 10;
            voltsDigits[1] = (voltsValue / 10) % 10;
            voltsDigits[2] = voltsValue % 10;

            // reset
            counter = 0;
            voltsSum = 0;
        }
    }

// PDB handler
void PDB0_IRQHandler(void) {
    ADC0_IRQHandler();
    PDB0->SC &= 0xfffffbf; // clear interrupt PDBIF
    PDB0->CH[0].S = 0x0;
}

int main(void) {

    SystemCoreClockUpdate();
    Intialize();

    while(1) {

    }
}

```